

Plugin de Correção de Erros

Como integrar a biblioteca em sites WordPress — três caminhos do mais simples ao mais formal, mais cuidados específicos do WordPress.

Resposta direta

Sim, funciona em WordPress — mas precisa de um pouco de "encanamento". O plugin *não* é um plugin WordPress nativo (não tem PHP, não tem painel de admin, não tem blocos Gutenberg). Ele é uma **biblioteca client-side** (CSS + JS puros) que você carrega na página.

Em WordPress, isso entra de três jeitos, do mais simples ao mais formal — abaixo cada um deles.

Caminho 1 — via “Insert Headers and Footers” (zero código)

Instale um plugin do tipo **WPCode**, **Insert Headers and Footers** ou **Code Snippets**. Cole o CSS do plugin (entre ▼ INÍCIO DO PLUGIN ▼ e ▲ FIM DO CSS DO PLUGIN ▲) num snippet de **header**, dentro de uma tag `<style>`. Cole o JS no **footer**, dentro de uma tag `<script>`. Em seguida, nas matérias, use o bloco “**HTML personalizado**” do Gutenberg para colar os `<div data-cde-bloco ...>`, `<div data-cde-resposta ...>`, etc.

É o caminho mais rápido para um piloto editorial — não requer acesso a código PHP nem ao tema.

Caminho 2 — embutir no tema

Se você (ou alguém da TI) tiver acesso ao tema-filho, salve dois arquivos: `cde.css` e `cde.js` na pasta do tema, e enfileire via `functions.php`:

```
add_action('wp_enqueue_scripts', function () {
    wp_enqueue_style(
        'cde',
        get_stylesheet_directory_uri() . '/cde.css'
    );
    wp_enqueue_script(
        'cde',
        get_stylesheet_directory_uri() . '/cde.js',
        [], null, true
    );
});
```

Os atributos `data-cde-*` continuam indo no conteúdo das matérias (via bloco “HTML personalizado” do Gutenberg).

Caminho 3 — empacotar como plugin WordPress de verdade

Wrapping em umas 15 linhas de PHP transforma a biblioteca num plugin instalável via o menu “Plugins → Adicionar novo”:

```
<?php
/**
 * Plugin Name: Correcao de Erros
 * Description: Marca correcoes, exclusoes e atualizacoes
 *              jornalisticas.
 * Version: 1.0
 */
add_action('wp_enqueue_scripts', function () {
    wp_enqueue_style(
        'cde',
        plugin_dir_url(__FILE__) . 'cde.css'
    );
    wp_enqueue_script(
        'cde',
        plugin_dir_url(__FILE__) . 'cde.js',
        [], null, true
    );
});
```

Coloque num zip junto com `cde.css`, `cde.js` e o cabeçalho do PHP — pronto, virou plugin WordPress oficial, instalável pelo painel.

Cuidados específicos do WordPress

1. wpautop (auto-format de parágrafos)

A função `wpautop` envolve parágrafos automaticamente em `<p>` e às vezes mexe em HTML solto. Os blocos `<div data-cde-*>` do plugin são autocontidos e não devem ser afetados, mas o melhor é sempre colar dentro do bloco **HTML personalizado** do Gutenberg, que preserva o markup tal como digitado.

2. Plugins de cache

Plugins como **WP Rocket**, **W3 Total Cache** e **LiteSpeed Cache** podem minificar o JS/CSS. O plugin não usa nada exótico, então roda bem com qualquer minificador padrão.

3. Conflitos com tema

O **namespace** `cde-*` evita conflito com a maioria dos temas. Se algum tema interferir, é provável que seja em `<details>/<summary>` ou `<blockquote>` — uma checagem rápida no inspector resolve.

4. Estrofes do Gutenberg

O editor de blocos às vezes “limpa” atributos HTML desconhecidos em blocos padrão (parágrafo, título, etc.). Para colar atributos `data-cde-*`, use sempre o bloco **“HTML personalizado”** — ele é o único que preserva HTML arbitrário. Alternativamente, o editor clássico (Classic Editor plugin) também funciona bem.

O que falta para um rollout sério em redação

Editores não-técnicos não vão escrever `data-cde-resposta` na mão. Para uso em produção, o ideal é construir, em cima desta biblioteca, **blocos Gutenberg customizados** — um para cada padrão:

- bloco **“Comunicar correção”** (factual / grafia / imprecisão / declaração)
- bloco **“Direito de resposta”**
- bloco **“Trecho excluído”**
- bloco **“Conteúdo novo”**

Cada bloco renderiza um pequeno formulário dentro do editor (com campos de *motivo*, *antes/depois*, *data*, etc.) e produz automático o HTML correto. É mais trabalho de implementação — exige um build setup com `@wordpress/scripts` — mas é o que destrava o uso editorial em escala.

Em resumo

Caminho	Quem faz	Quando usar
1. Insert Headers and Footers	Editor de site / SEO	Piloto rápido, sem código

2. Embutir no tema-filho	Dev / TI	Site próprio, controle total
3. Plugin WordPress empacotado	Dev (15 linhas PHP)	Distribuir entre vários sites
4. Blocos Gutenberg customizados	Dev (build setup)	Redação editorial em produção

O que está pronto neste protótipo: caminhos 1, 2 e 3 funcionam como descritos — é copiar e colar o CSS/JS do arquivo `correcao-de-erros-plugin.html`. **O que ainda exige desenvolvimento:** caminho 4 (blocos Gutenberg), que é o passo seguinte para um *rollout* editorial em escala.

Documento gerado a partir das informações discutidas durante o desenvolvimento do protótipo de pesquisa — Lívia Vieira, abril de 2026.